UNIVERSIDAD EAFIT
DEPARTAMENTO DE INFORMÁTICA Y SISTEMAS
ST1630 — SISTEMAS INTENSIVOS EN DATOS

PROYECTO FINAL — ENTREGABLE 1
Diagnóstico, Requisitos y Arquitectura

Sistema de Pagos Inmediatos Interoperables (Bre-B) y Detección de Fraude Transaccional en Tiempo Real

Dominio Elegido: Fintech Colombiana — Pagos en Tiempo Real y Prevención de Fraude

Equipo de Trabajo:

Nombre Completo	Código EAFIT	Rol en el Proyecto
Samuel Arango	1027802460	Lead de Ingesta y Streaming — Kafka, Spark Structured Streaming
Mateo Sanz Medina	1034988889	Lead de Almacenamiento — Lakehouse Delta Lake y NoSQL MongoDB
Nathalia Valentina Cardoza Azuaje	4992531	Lead de DataOps — Orquestación Airflow y Plataforma Docker
Sebastian Andres Medina Cabezas	1000317163	Lead de Consumo y Gobernanza — API de scoring, BI y linaje de datos

Profesor: Andrés Sacre Alzate
Semestre 2026-2 — Versión 1.0
Medellín, Colombia

Sección 1. Definición del Problema

1.1 Contexto de Industria y Caso de Negocio

En Colombia, la adopción masiva de billeteras digitales (Nequi, Daviplata, Dale!) y la entrada en vigor del Sistema de Pagos Inmediatos (SPI) 'Bre-B', regulado y operado por el Banco de la República bajo la Circular Reglamentaria Externa DSP-465, transforman de forma radical la dinámica transaccional del país. El sistema exige la interoperabilidad en tiempo real entre entidades bancarias, billeteras digitales y cooperativas financieras mediante el uso de identificadores unificados ('llaves': número de celular, documento de identidad, correo electrónico o código alfanumérico).

Bajo esta nueva infraestructura nacional, las transferencias interbancarias deben liquidarse y acreditarse en la cuenta del destinatario en menos de 20 segundos, operando en modalidad ininterrumpida 24/7/365. Si bien este avance reduce drásticamente la fricción en la economía popular y el comercio digital, introduce una vulnerabilidad crítica: la velocidad de consumación del fraude supera la capacidad de respuesta de los sistemas analíticos tradicionales. Modalidades delictivas como el Account Takeover (toma de control de cuenta por credenciales filtradas), el phishing asistido por ingeniería social y el reclutamiento masivo de 'cuentas puente' o mulas de dinero permiten a redes criminales dispersar fondos ilícitos entre 5 o más entidades financieras en cuestión de minutos, retirando el dinero en efectivo en cajeros automáticos o puntos corresponsales antes de que se genere una alerta manual.

1.2 El Dolor Financiero y Operacional

El modelo analítico tradicional de la banca colombiana se basa en el procesamiento por lotes (batch processing) que consolida transacciones durante cortes nocturnos (ciclos compensatorios ACH tradicionales). Este rezago de 12 a 24 horas resulta obsoleto frente al ecosistema Bre-B: una vez la transacción es autorizada y acreditada, la irreversibilidad de los pagos inmediatos impide recuperar el capital, transfiriendo la pérdida neta a la entidad financiera o al usuario y acarreando sanciones regulatorias por parte de la Superintendencia Financiera de Colombia (SFC) por fallas en la gestión del riesgo operativo y de ciberseguridad (Circular Básica Jurídica, Capítulo XXIII).

1.3 Preguntas Analíticas y Operacionales (Específicas y Medibles)

Para mitigar este riesgo sin degradar la experiencia de usuario (evitando falsos positivos que bloqueen transferencias legítimas), el sistema debe responder en tiempo de ejecución las siguientes preguntas cuantitativas:

1. ¿Supera el usuario su frecuencia transaccional crítica en ventanas cortas?
   Métrica y Umbral: Número de transferencias salientes hacia destinatarios nuevos en ventanas móviles de 5 minutos. Se genera alerta cuando supera el mayor entre el percentil 95 (P95) del historial del usuario y 4 transferencias.
2. ¿Existe un patrón de dispersión rápida hacia cuentas mulas?
   Métrica y Umbral: Detección de cuentas receptoras que reciben más de 10 transferencias de diferentes ordenantes en una ventana de 10 minutos y dispersan más del 85% del saldo recibido en menos de 3 minutos hacia otras llaves o billeteras.
3. ¿Presenta la transacción una anomalía geoespacial y de dispositivo concurrente?
   Métrica y Umbral: Velocidad de desplazamiento aparente entre el inicio de sesión anterior y la transferencia actual (alerta inmediata si la velocidad implícita supera los 800 km/h, sugiriendo uso de túneles VPN o suplantación remota). La ubicación y hora de la sesión previa se conservan en el perfil del usuario (RF8).
4. ¿Cuál es la tasa de aprobación y rechazo transaccional agregada por canal en tiempo real?
   Métrica y Umbral: Ratio porcentual de transacciones marcadas como fraudulentas respecto al volumen total en ventanas deslizantes de 1 minuto por entidad participante del ecosistema. Se calcula sobre las decisiones de riesgo persistidas por el Job 2 (RF13), agrupadas por minuto, entidad y canal.

1.4 Usuarios del Sistema y Decisiones Operativas

- Motor de Autorización Transaccional en Línea (Sistema Operacional): consulta de forma síncrona el score de riesgo vigente del ordenante, disponible en MongoDB a través de una API con latencia p99 inferior a 300 ms, dentro de un presupuesto total inferior a 1 segundo, para decidir entre tres acciones: aprobar la transferencia, solicitar segundo factor dinámico biométrico (step-up authentication) o rechazar y congelar preventivamente el saldo. La frescura de esa evaluación depende del pipeline en streaming, que la actualiza en menos de 3 segundos desde que el evento llega a Kafka (RNF-02).
- Separación de rutas de latencia: Es necesario precisar que el motor de autorización no espera la finalización del pipeline de streaming. La arquitectura desacopla deliberadamente dos rutas con SLA distintos: (i) una ruta síncrona de decisión, que resuelve en menos de 300 ms leyendo el perfil de riesgo pre-calculado desde el almacén NoSQL de baja latencia, y (ii) una ruta asíncrona de actualización, en la que Spark Structured Streaming recalcula las agregaciones de ventana y refresca ese perfil en menos de 3 segundos. Esta separación, equivalente al patrón de pre computación de vistas materializadas, es lo que permite cumplir simultáneamente el SLA del switch Bre-B (menos de 1 s) y la complejidad analítica de las ventanas deslizantes de 5 minutos, que sería imposible de calcular dentro de la ventana síncrona de autorización.
- Analistas de la Unidad de Fraude y Riesgo Operativo (Usuarios Humanos): Monitorean tableros en tiempo real para confirmar redes de estafa complejas, generar reportes de operaciones sospechosas (ROS) para la UIAF y calibrar los umbrales de las reglas de inferencia.
- Oficiales de Cumplimiento Normativo: Auditan el linaje y traza forense de cada bloqueo transaccional ante requerimientos y procesos sancionatorios de la Superintendencia Financiera de Colombia.

1.5 Justificación de una Arquitectura de Datos Distribuida frente a un RDBMS

Un motor relacional monolítico tradicional (como PostgreSQL o MySQL) resulta inviable para este problema debido a las siguientes restricciones técnicas:

a) Contención de Concurrencia y Bloqueos de Escritura (Lock Contention): Sobre un caudal nominal de 5.000 transacciones por segundo, durante fechas críticas del comercio colombiano (días de quincena, pagos de nómina, Cyberlunes, fin de año) el sistema debe absorber picos de hasta 25.000 transacciones por segundo (TPS). En un RDBMS tradicional, las actualizaciones de índices B-Tree y tablas maestras generan contención de cerraduras a nivel de fila y página, degradando la latencia de escritura a niveles inaceptables (> 2.000 ms).
b) Cálculo de Agregaciones Temporales en Memoria (Windowing): Evaluar el comportamiento transaccional en ventanas deslizantes de 1 a 10 minutos para millones de usuarios activos exige mantener estados en memoria particionados horizontalmente, operación que saturaría la memoria de un servidor relacional único mediante escaneos de índices repetitivos (table scans).
c) Desacoplamiento y Disponibilidad: Un fallo en la base de datos de análisis no puede detener la recepción de pagos del switch de Bre-B. Se requiere un búfer distribuido de persistencia inmutable que desacople la velocidad de ingestión de la velocidad de transformación analítica.

Sección 2. Requisitos Funcionales y No Funcionales

2.1 Requisitos Funcionales

- RF1 (Ingesta continua): El sistema debe ingerir de manera continua y asíncrona eventos transaccionales emitidos por el switch central de Bre-B y canales digitales en formato JSON con una latencia de transporte menor a 300 ms.
- RF2 (Validación de esquema): El sistema debe validar la integridad estructural y sintáctica de cada evento entrante contra un esquema formal, enrutando los mensajes corruptos o desalineados hacia un tópico de mensajes muertos (Dead Letter Queue - DLQ) para auditoría.

- RF3 (Agregación temporal): El sistema debe computar agregaciones temporales con PySpark Structured Streaming: (a) conteo de eventos, suma de montos y variación de llaves de destino por ordenante, en ventanas deslizantes de 5 minutos con saltos de 1 minuto; (b) cantidad de ordenantes distintos y monto recibido por cuenta receptora, en ventanas de 10 minutos con saltos de 1 minuto, para detectar dispersión hacia cuentas mulas.
- RF4 (Enriquecimiento en streaming): El sistema debe enriquecer el flujo de eventos en tiempo real cruzando el identificador del ordenante con el perfil histórico de comportamiento del cliente almacenado en la capa de baja latencia antes de generar el score de riesgo.
- RF5 (Persistencia Histórica en Bronze): El sistema debe persistir el 100% de los eventos transaccionales crudos ingeridos desde Kafka en la capa Bronze sobre tablas Delta Lake en formato Parquet particionadas por fecha (year, month, day), conservando el payload JSON original sin modificaciones para fines de inmutabilidad y auditoría forense.
- RF6 (Deduplicación y Calidad en Silver): El sistema debe sanear, tipar y deduplicar en streaming (Job 3) las transacciones de Bronze hacia la capa Silver de Delta Lake, mediante el identificador único 'transaction_uuid' y un watermark de 10 minutos, aplicando validaciones de esquema estrictas (Schema Enforcement) y normalización temporal a UTC. Los duplicados tardíos (posteriores a la ventana del watermark) y los eventos reinyectados desde la DLQ se absorben con un MERGE por transaction_uuid que inserta solo si el registro no existe.
- RF7 (Consolidación Dimensional en Gold): El sistema debe construir y actualizar cada 15 minutos, mediante un job batch (Job 4) orquestado por Airflow, un modelo dimensional en estrella (Star Schema) en la capa Gold de Delta Lake, articulado alrededor de la tabla de hechos atómica 'Fact_Transferencias' y dimensiones conformadas ('Dim_Usuario', 'Dim_Entidad_Financiera', 'Dim_Canal', 'Dim_Tiempo'), permitiendo responder consultas analíticas con tiempos de respuesta menores a 5 segundos.
- RF8 (Persistencia Operacional NoSQL de Baja Latencia): El sistema debe sincronizar y almacenar el perfil de riesgo transaccional actualizado de cada usuario/cuenta (contadores en ventanas móviles, saldo medio, dispositivos validados) en MongoDB, particionada por 'account_id', garantizando lecturas operacionales puntuales por clave de partición con latencia p50 inferior a 15 milisegundos y p99 inferior a 50 milisegundos para el motor de inferencia. La colección admite dos escritores sobre campos disjuntos del mismo documento: el Job 2 actualiza, mediante operaciones $set, el subdocumento contadores (ventanas móviles) y el subdocumento ultima_sesion (marca de tiempo, coordenadas y dispositivo del último evento de sesión de la app), y Airflow carga el perfil histórico (saldo medio, dispositivos validados) desde Gold sobre su propio subdocumento, de modo que ninguna escritura sobrescribe los campos de la otra.
- RF9 (Orquestación de dependencias): el sistema debe orquestar mediante un DAG de Apache Airflow la ejecución automática de los jobs batch del pipeline (consolidación dimensional de Gold, sincronización del perfil de riesgo hacia MongoDB y mantenimiento del Lakehouse), disparando cada tarea únicamente cuando todas sus dependencias upstream hayan finalizado en estado exitoso, con hasta 3 reintentos automáticos con retroceso exponencial ante fallos transitorios. Los jobs de streaming (1 a 3) corren de forma continua y Airflow sólo supervisa su salud.
- RF10 (Emisión de alertas operativas): El sistema debe publicar toda transacción cuyo score de riesgo supere el umbral crítico en el tópico Kafka 'fraud.alerts.critical' en menos de 1 segundo desde el cierre del micro-batch que la evaluó, incluyendo 'transaction_uuid', score numérico, lista de reglas disparadas y nivel de severidad, habilitando la retención preventiva de fondos por parte del sistema operacional.
- RF11 (Exposición analítica a usuarios de negocio): El sistema debe exponer las tablas de la capa Gold como vistas SQL consultables vía Apache Spark SQL hacia tableros de Metabase, permitiendo a los analistas de la Unidad de Fraude responder las cuatro preguntas de negocio de la Sección 1.3 sin acceso directo a Bronze o Silver: las preguntas 1 a 3 desde Fact_Transferencias y sus dimensiones, y la pregunta 4 desde la tabla risk_decisions (RF13).
- RF12 (Reproceso de eventos rechazados): El sistema debe permitir el reproceso controlado de los eventos acumulados en el tópico 'breb.dlq.invalid_events' mediante una tarea de Airflow ejecutable bajo demanda,

reinyectándolos al pipeline una vez corregido el esquema y sin generar registros duplicados en la capa Silver gracias a la deduplicación por 'transaction_uuid'.

- RF13 (Persistencia de decisiones de riesgo): El sistema debe persistir en cada micro-batch del Job 2, en la tabla Delta risk_decisions de la capa Gold, una fila por transacción evaluada con transaction_uuid, event_ts, entidad, canal, risk_score, severidad, reglas disparadas y decisión, mediante MERGE por transaction_uuid, de modo que la tasa de transacciones marcadas se calcule en ventanas de 1 minuto por entidad.

2.2 Requisitos No Funcionales

Requisitos cuantitativos fundamentados en los conceptos del curso (Escalabilidad, Latencia, Garantías, CAP, Tolerancia a Fallos y Linaje):

- RNF-01 (Escalabilidad de Ingestión): El subsistema de ingestión debe procesar de forma sostenida un caudal nominal de 5.000 eventos/segundo y absorber picos de tráfico de hasta 25.000 eventos/segundo durante eventos de alta transaccionalidad nacional (ej. día de pago quincenal) sin pérdida de paquetes ni incremento de latencia superior al 15%.

  Los 25.000 eventos/s son un objetivo de diseño dimensionado sobre el particionado de Kafka; las pruebas del E2 se ejecutan a 500–1.000 eventos/s (ver ADR 06).

- RNF-02 (Latencia de Extremo a Extremo en Streaming): el tiempo entre la publicación del evento en el broker de Kafka y la disponibilidad del resultado de la ventana enriquecida para el motor de decisiones (tras el upsert a MongoDB y la emisión de alertas) debe ser inferior a 3 segundos en el percentil 99. El transporte desde la fuente hasta Kafka se mide aparte (menos de 300 ms, RF1).
- RNF-03 (Garantía de Entrega y Semántica): El flujo de transporte y procesamiento debe garantizar semántica de entrega At-Least-Once entre los productores y el clúster de streaming, complementada con mecanismos de deduplicación determinista basados en el identificador único de transacción (transaction_uuid) para lograr consistencia efectiva de estado equivalente a Exactly-Once en Silver (Job 3), MERGE por clave en Gold (Job 4) y upsert por account_id en MongoDB. Un mensaje duplicado por Kafka o una tarea reintentada no altera el estado final.
- RNF-04 (Disponibilidad y Posición en el Teorema CAP): El subsistema operacional de almacenamiento rápido (NoSQL) debe operar formalmente como un sistema CP (Consistente y Tolerante a Particiones) bajo el Teorema CAP. Se prioriza la Consistencia Fuerte sobre la Disponibilidad total (AP), debido a que en transferencias financieras inmediatas (Bre-B) una lectura desactualizada de un saldo o de un perfil con sospecha de fraude provocaría la autorización indebida de transacciones no recuperables. El sistema debe garantizar un SLA de disponibilidad del 99.9% (menos de 8.76 horas de indisponibilidad anual), aplicando políticas de lectura mayoritaria (readConcern: majority) y escritura mayoritaria (w: majority).
- RNF-05 (Tolerancia a Fallos e Integridad Transaccional ACID en Lakehouse): El almacenamiento analítico (Bronze, Silver, Gold) debe garantizar atomicidad, consistencia, aislamiento y durabilidad (ACID) a través del registro transaccional serializado de Delta Lake (_delta_log). Ante la caída de un nodo ejecutor de Spark o una desconexión en el almacenamiento de objetos, el sistema no debe dejar escrituras parciales o archivos huérfanos (dirty writes), garantizando recuperación determinista mediante aislamiento de instantáneas (Snapshot Isolation) y puntos de control Parquet (checkpoints) generados automáticamente cada 10 commits.
- RNF-06 (Trazabilidad y Linaje de Datos): El sistema debe permitir que un oficial de cumplimiento reconstruya el linaje de cualquier transacción, desde el evento crudo recibido en Kafka hasta su registro en la capa Gold, en menos de 5 minutos. Se logra con dos mecanismos nativos del pipeline: el identificador transaction_uuid, que se propaga sin cambios por las cinco capas y permite consultar un mismo evento en cada una, y las coordenadas de origen que cada registro de Bronze conserva del bróker (kafka_topic, kafka_partition, kafka_offset, ingestion_timestamp). Adicionalmente, el log transaccional de Delta Lake

(_delta_log) permite auditar por Time Travel el estado exacto de los datos en el momento en que se tomó una decisión de bloqueo, mediante SELECT ... VERSION AS OF <n>.

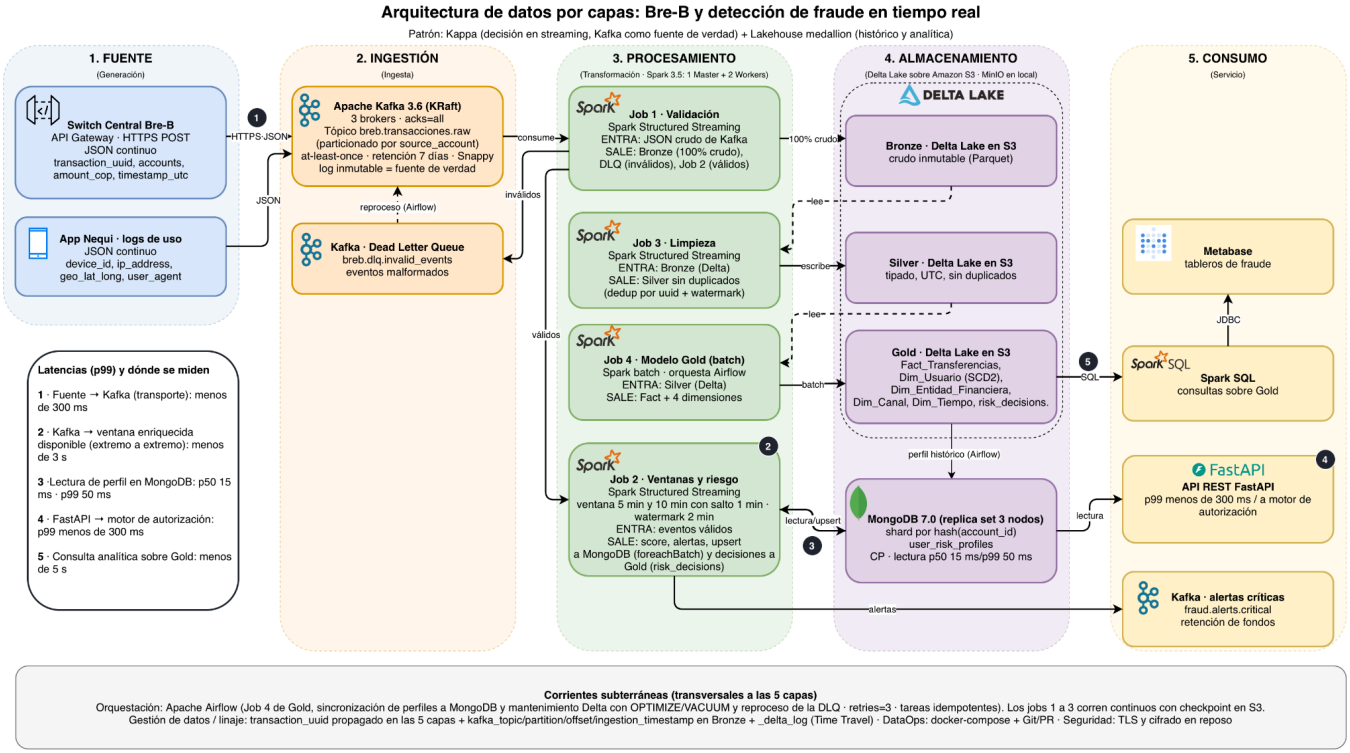
- RNF-07 (Frescura de los datos): un evento publicado en Kafka debe estar disponible en Silver en menos de 60 segundos y en Gold en menos de 15 minutos, ciclo del job batch orquestado por Airflow. Este requisito mide la frescura del dato y es independiente del tiempo de respuesta de las consultas sobre Gold (menos de 5 segundos, RF7).

2.3 Trazabilidad requisito-diagrama

Requisito	Componente del diagrama
RF1, RNF-01	Switch Bre-B y App Nequi → Apache Kafka (marcador 1)
RF2	Job 1 y tópico DLQ
RF3	Job 2 (ventanas de 5 y 10 min, con salto de 1 min, watermark de 2 min)
RF4	Flecha de lectura entre Job 2 y MongoDB
RF5	Job 1 → Bronze
RF6	Job 3 → Silver
RF7	Job 4 → Gold (marcador 5)
RF8, RNF-04	MongoDB: upsert del Job 2 y perfil histórico desde Gold (marcador 3)
RF9	Franja de orquestación y Job 4 (Apache Airflow)
RF10	Job 2 → tópico Kafka de alertas críticas
RF11	Gold (Fact_Transferencias y risk_decisions) → Spark SQL → Metabase
RF12	Flecha punteada DLQ → Kafka (reproceso con Airflow)
RF13	Job 2 → risk_decisions (Gold)
RNF-02	Kafka → Job 2 → MongoDB (marcador 2)
RNF-03	Kafka (at-least-once) con idempotencia en Jobs 3 y 4 y en MongoDB
RNF-05	Capa Delta Lake (_delta_log)
RNF-06	Franja de gestión de datos y linaje
RNF-07	Silver y Gold

Sección 3. Diagrama de Arquitectura de Datos por Capas

Enlace: https://drive.google.com/file/d/1-GXf4H6ko0-zz_um0fnQaJM5uNrHcffN/view?usp=sharing



- Patrón arquitectónico: El sistema combina una arquitectura Kappa para la decisión de fraude y un Lakehouse con capas medallion para el histórico y la analítica. Kafka es el log inmutable y fuente de verdad: la lógica de detección vive una sola vez, en Spark Structured Streaming, y si hay que reprocesar se relee el log o la capa Bronze. Se descarta Lambda porque mantener la misma lógica en un camino batch y otro streaming duplica el código y la deuda operativa.
- Lectura del diagrama: Las fuentes publican JSON en Kafka con garantía *at-least-once*. El Job 1 valida cada evento: aterriza el 100% del crudo en Bronze, desvía los inválidos a la DLQ y entrega los válidos al Job 2, que calcula ventanas de 5 y 10 minutos, enriquece con el perfil de MongoDB, actualiza los contadores, emite alertas y persiste cada decisión en risk_decisions. El Job 3 limpia y deduplica Bronze hacia Silver en streaming; el Job 4, orquestado por Airflow, construye el Star Schema de Gold por lotes. Airflow también carga el perfil histórico desde Gold a MongoDB y ejecuta el mantenimiento de las tablas Delta. El consumo se divide en analítica (Spark SQL y Metabase sobre Gold) y operación en línea (FastAPI sobre MongoDB y alertas en Kafka).
- Latencias y puntos de medición: Los marcadores numerados del diagrama indican dónde se mide cada límite, y no son acumulables sin aclarar el punto de partida. (1) El transporte, desde que la fuente envía el evento hasta que Kafka lo confirma, es menor a 300 ms (RF1). (2) La propagación de extremo a extremo en streaming, desde que el evento queda publicado en el broker hasta que el resultado de la ventana enriquecida está disponible para el motor de decisiones, es menor a 3 s en el percentil 99 (RNF-02); como cota, el recorrido total desde la fuente es de unos 3,3 s. (3) La lectura operacional del perfil de riesgo en MongoDB se resuelve con p50 inferior a 15 ms y p99 inferior a 50 ms (RF8). (4) El servicio FastAPI responde al motor de autorización con p99 inferior a 300 ms, incluyendo esa lectura y la evaluación del motor de reglas, holgadamente dentro del presupuesto de 1 segundo del switch. (5) Las consultas analíticas sobre Gold responden en menos de 5 s (RF7); este valor mide el tiempo de respuesta de una consulta, no la

frescura del dato, que se define aparte en el RNF-07. Salvo el p50 indicado en el punto (3), todos los límites se evalúan en el percentil 99.

3.1 Ficha Técnica Completa de las 5 Capas de la Arquitectura

- Capa 1 (Fuentes de Datos - Generación): Switch Central Bre-B (API Gateway, HTTPS POST, JSON continuo con ‘transaction_uuid’, ‘accounts’, ‘amount_cop’, ‘timestamp_utc’) y Logs de Navegación App Nequi (JSON continuo con ‘device_id’, ‘ip_address’, ‘geo_lat_long’, ‘user_agent’, account_id, timestamp_utc, event_type). Ambas fuentes publican en Kafka.
- Capa 2 (Ingestión de Datos): Clúster Apache Kafka 3.6 (3 Brokers en modo KRaft). Tópicos particionados por hash: 'breb.transacciones.raw' (particionado por source_account, retención de 7 días, garantía at-least-once, ‘acks=all’, compresión Snappy) y 'breb.dlq.invalid_events' como DLQ (Dead Letter Queue). El log inmutable es la fuente de verdad.
- Capa 3 (Procesamiento Distribuido): PySpark Structured Streaming 3.5 (1 Master, 2 Workers).
    - Job 1, Validación (Streaming): entra el JSON crudo de Kafka; sale el 100% a Bronze, los inválidos a la DLQ y los válidos al Job 2.
    - Job 2, Ventanas y riesgo (Streaming): Ventanas deslizantes de 5 y 10 min con salto de 1 minuto y watermarking de 2 minutos; entran los eventos válidos, que se enriquecen con el perfil de MongoDB; sale el score de riesgo, las alertas al tópico ‘fraud.alerts.critical’, las decisiones a risk_decisions (Gold) y el upsert de contadores a MongoDB (‘foreachBatch’).
    - Job 3, Limpieza (streaming): entra Bronze; sale Silver con esquema estricto, UTC y deduplicado por ‘transaction_uuid’ con watermark de 10 minutos.
    - Job 4, Modelo Gold (batch, orquestado por Airflow): entra Silver; salen ‘Fact_Transferencias’ y las 4 dimensiones.
    - Los Jobs 1 a 3 corren de forma continua con checkpoint en S3.
- Capa 4 (Almacenamiento Lakehouse & NoSQL): Formato de tabla abierta Delta Lake sobre almacenamiento de objetos S3/MinIO estructurado en capas:
    - Bronze: 's3://lakehouse-bronze/transactions/' , tabla Delta Lake en modo append-only (archivos de datos en formato Parquet con log transaccional _delta_log), particionada por year/month/day. Conserva el payload JSON original íntegro en la columna 'raw_payload' más metadatos de ingesta (kafka_topic, kafka_partition, kafka_offset, ingestion_timestamp). Se elige tabla Delta y no Parquet crudo porque el sink de Delta registra en el _delta_log el identificador de la consulta y el número de micro-batch de cada escritura, lo que hace idempotente la escritura de Bronze: si un ejecutor cae a mitad de un micro-batch, el reintento no deja datos parciales visibles (los archivos sin confirmar los elimina VACUUM).
    - Silver: 's3://lakehouse-silver/transactions_cleansed/' (Delta Lake deduplicado por transaction_uuid con Z-ORDER BY source_account).
    - Gold: 's3://lakehouse-gold/analytics/' estructurado en Star Schema: Tabla de Hechos 'Fact_Transferencias' (grano atómico por transacción), las dimensiones 'Dim_Usuario' (SCD Tipo 2), 'Dim_Entidad_Financiera', 'Dim_Canal' y 'Dim_Tiempo' y la tabla risk_decisions, escrita por el Job 2.
    - Componente NoSQL Operacional: MongoDB 7.0 Clustered (Replica Set 3 nodos, Sharding por hash de ‘account_id’) alojando la colección 'user_risk_profiles' para consultas operacionales de perfiles con latencia p50 menor a 15 ms y p99 menor a 50 ms. Dos escritores: el Job 2 (contadores de ventana y última sesión) y Airflow (perfil histórico desde Gold).
- Capa 5 (Consumo y Exposición): Consultas SQL analíticas vía Apache Spark SQL sobre Gold, conectadas por JDBC a tableros en Metabase. Consultas operacionales en tiempo real vía API REST construida en FastAPI (Python) que lee MongoDB, consumida por el switch de autorización con latencia extremo a extremo p99 inferior a 300 ms (incluyendo la lectura del perfil en MongoDB y la evaluación del motor de

reglas), holgadamente dentro del presupuesto de 1 segundo del switch. Alertas críticas publicadas al tópico Kafka 'fraud.alerts.critical' para retención inmediata de fondos.

- Capa transversal A (Orquestación y DataOps): Apache Airflow 2.8 desplegado con LocalExecutor y metastore en PostgreSQL 15. Un DAG principal breb_lakehouse_pipeline con schedule */15 * * * * coordina las tareas batch del pipeline mediante SparkSubmitOperator y DockerOperator. La orquestación no interviene en el flujo por evento (que es responsabilidad continua de Spark Structured Streaming), sino que gobierna el ciclo de vida de los jobs batch, el mantenimiento del Lakehouse (OPTIMIZE, VACUUM) y la supervisión de salud del job de streaming mediante un sensor de latencia de consumo (consumer lag). Un segundo DAG, reprocess_dlq, sin programación, se dispara bajo demanda para el reproceso de la DLQ (RF12).
- Capa Transversal B (Gobernanza y Linaje): Trazabilidad con mecanismos nativos del pipeline, sin servidores de metadatos adicionales: propagación del transaction_uuid por las cinco capas, coordenadas de origen del bróker (kafka_topic, kafka_partition, kafka_offset, ingestion_timestamp) en cada registro de Bronze, e historial de versiones de Delta Lake (DESCRIBE HISTORY y Time Travel) en las tres capas del Lakehouse.

Sección 4. Justificación de Decisiones Técnicas (ADRs)

Cada decisión técnica no trivial se documenta bajo el formato formal de Architecture Decision Record (ADR) con sus 4 componentes: Contexto, Alternativas consideradas, Decisión y Consecuencias.

ADR 01: Elección del Motor de Ingesta y Streaming Distribuido

Contexto: El ecosistema de transferencias inmediatas de Bre-B genera un flujo continuo de transacciones financieras emitidas desde múltiples entidades participantes de manera concurrente. Se requiere una plataforma de mensajería distribuida capaz de absorber ráfagas masivas de eventos, ofrecer almacenamiento intermedio desacoplado, proveer ordenamiento estricto a nivel de cuenta bancaria/llave y garantizar tolerancia total a fallos de nodos sin pérdida de transacciones.

Alternativas consideradas:

- AWS Kinesis Data Streams: Servicio administrado de alta calidad, pero descartado debido a los límites estrictos de throughput por shard (1 MB/s escritura), la complejidad de re-sharding dinámico durante picos repentinos y el vendor lock-in que limita despliegues híbridos bancarios.
- RabbitMQ: Bróker de mensajería tradicional AMQP, descartado debido a que su arquitectura no fue diseñada para retención histórica de eventos en disco a largo plazo ni para permitir que múltiples consumidores independientes lean el flujo a diferentes ritmos sin saturar la RAM.
- Apache Kafka: Plataforma de streaming de eventos distribuida basada en un commit log inmutable particionado horizontalmente y almacenado en disco.

Decisión: Se selecciona Apache Kafka como columna vertebral de ingestión. La decisión se fundamenta en su capacidad de particionado por clave (source_account) para garantizar orden estricto, su modelo de consumo pull-based basado en offsets que desacopla a Spark de la ingestión, y la configuración de productores con acks=all y factor de replicación RF >= 3, asegurando semántica At-Least-Once sin pérdida de datos.

Consecuencias: Se obtiene rendimiento masivo de millones de eventos por segundo con latencias de milisegundos y retención desacoplada en disco. Como contrapartida (trade-off), se asume una mayor complejidad operativa en la gestión de metadatos (KRaft), afinamiento de JVM y administración de volúmenes de almacenamiento en bloque.

ADR 02: Paradigma de Procesamiento en Tiempo Real (Spark Structured Streaming vs. Flink vs. Batch)

Contexto: El sistema debe evaluar reglas de detección de fraude sobre ventanas deslizantes de tiempo (5 minutos) y enriquecer las transacciones de Bre-B antes de persistirlas en el Lakehouse (Delta Lake). Se debe elegir el motor de procesamiento que equilibre la latencia requerida por el negocio (menor a 3 segundos) con la estabilidad y compatibilidad con el resto del pipeline analítico.

Alternativas consideradas:

- Batch Puro Programado (PySpark vía cron/Airflow cada 5 min): Descartado categóricamente. Iniciar aplicaciones Spark cada 5 minutos introduce una sobrecarga prohibitiva de arranque de JVM y asignación de recursos, generando una ceguera temporal inaceptable para detener fraudes en curso.
- Apache Flink (Streaming Nativo Event-by-Event): Motor de procesamiento continuo con latencias inferiores a 50 ms. Aunque excelente en su dominio, fue descartado por tres razones: la sobrecarga cognitiva de mantener dos motores de cálculo diferentes (Flink para streaming y Spark para el Lakehouse), la ausencia de una integración nativa con Delta Lake equivalente a la de Spark, y una curva de aprendizaje incompatible con el cronograma a S16.
- PySpark Structured Streaming (Micro-batching de baja latencia): Motor unificado que procesa flujos continuos como tablas de datos infinitas utilizando micro-lotes de alta frecuencia (intervalos de disparo de 1 segundo).

Decisión: Se selecciona PySpark Structured Streaming operando con micro-batch triggers de 1 segundo. Se respalda en su soporte maduro de agregaciones en ventanas deslizantes con watermarking automático de 2 minutos para evitar saturación de memoria, su integración nativa para escrituras transaccionales ACID en Delta Lake mediante checkpoints distribuidos, y la unificación de código entre streaming y batch.

Consecuencias: Se logra simplicidad arquitectónica (un solo motor de cómputo para todo el sistema) y recuperación automática ante fallas. Se sacrifica la latencia a nivel de submilisegundos de Flink; sin embargo, para el SLA de detección definido en el RNF-02 (menos de 3 segundos en el percentil 99), la latencia de 1 s de Spark es idónea y elimina duplicidad operativa.

ADR 03: Elección del Formato de Tabla Abierta (Open Table Format) y Modelo Dimensional en Capa Gold

Contexto: El volumen proyectado del sistema Bre-B supera los 15 millones de transacciones diarias. Se requiere un formato de almacenamiento analítico sobre object storage desacoplado (S3/MinIO) que permita lecturas concurrentes de alta velocidad para tableros de BI sin degradar las escrituras continuas del pipeline de streaming. Además, por exigencias normativas de la Superintendencia Financiera de Colombia (SFC), es obligatorio auditar y reproducir el estado exacto de los datos en cualquier instante histórico (Time Travel).

Alternativas consideradas:

- Archivos Parquet Crudos sobre S3 (Data Lake Clásico): Descartado. No ofrece garantías transaccionales ACID; las escrituras concurrentes de streaming y batch provocan lecturas inconsistentes (dirty reads) y fallas por archivos huérfanos. Carece de mecanismos de evolución de esquemas y viajes en el tiempo.
- Data Warehouse Relacional Tradicional (PostgreSQL / Redshift): Descartado debido al acoplamiento entre almacenamiento y cómputo, costos elevados de licenciamiento en la nube ante volúmenes masivos y lentitud en la carga continua de micro-batches de streaming.
- Apache Iceberg: Excelente formato de tabla abierta con metadata jerárquica desacoplada. Sin embargo, su integración operativa con PySpark Structured Streaming en entornos locales y contenedores Docker presenta mayor complejidad de configuración frente a Delta Lake.
- Delta Lake sobre Almacenamiento de Objetos: Formato de tabla abierta optimizado nativamente para Apache Spark que implementa un log transaccional serializado (_delta_log).

Decisión: Se selecciona Delta Lake como formato unificado de tabla abierta para las tres capas del Lakehouse (Bronze, Silver y Gold), adoptando un Modelo Dimensional en Estrella (Star Schema) en la capa Gold. La decisión se fundamenta en:

- Transacciones ACID y Aislamiento de Instantáneas: Garantiza que los analistas de negocio consulten la capa Gold sin colisionar con los micro-batches que actualizan Silver continuamente.
- Time Travel y Auditoría Regulatoria: La retención de versiones del log transaccional permite ejecutar consultas históricas (VERSION AS OF) requeridas por entes de control para investigar fraudes pasados.
- Schema Enforcement y Evolution: Impide que datos corruptos alteren las tablas Gold y permite agregar nuevas columnas de métricas de riesgo sin reconstruir la tabla histórica.
- Adecuación del Star Schema: La estructura con la tabla Fact_Transferencias y dimensiones conformadas simplifica radicalmente las consultas SQL de BI y aprovecha el algoritmo de Z-Ordering de Delta Lake para reducir el escaneo de datos en más de un 80%.

Consecuencias: Se obtiene consistencia transaccional absoluta sobre almacenamiento de objetos económicos, capacidad de auditoría forense histórica y alta velocidad de consulta. Como contrapartida (trade-off), se presenta el fenómeno de acumulación de archivos pequeños (small files problem) por las micro-escrituras de streaming, lo cual exige programar tareas periódicas de mantenimiento (OPTIMIZE para compactación y VACUUM para depuración de snapshots antiguos).

ADR 04: Selección del Componente NoSQL Libre y Posición en el Teorema CAP

Contexto: Durante la fase de procesamiento en tiempo real, cada transacción de Bre-B debe enriquecerse con el perfil del usuario (número de transferencias recientes, saldo medio diario, dispositivos validados). Consultar la capa Gold o Silver de Delta Lake en S3 introduce latencias de escaneo de 1 a 4 segundos, lo que viola el SLA del motor de inferencia de fraude (menos de 50 ms). Se requiere un almacén de datos no relacional (NoSQL) de acceso por clave-valor o documental de lectura ultra-rápida.

Alternativas consideradas:

- Apache Cassandra (Modelo Wide-Column / AP): Motor distribuido de altísima disponibilidad y escalabilidad horizontal de escritura. Sin embargo, Cassandra sigue el principio AP (Disponibilidad y Tolerancia a Partición) bajo consistencia eventual. En una fintech de pagos inmediatos, la consistencia eventual introduce un riesgo fatal: si un atacante dispersa dinero desde dos terminales simultáneas y el nodo réplica aún no ha propagado el saldo o el bloqueo preventivo, ambas transacciones serían autorizadas fraudulentamente.
- Neo4j (Base de Datos de Grafos): Excepcional para descubrir anillos de fraude complejos en profundidad. No obstante, su latencia de escritura y la complejidad de particionado distribuido (sharding) la hacen inviable como almacén operacional de alta concurrencia (25.000 TPS).
- MongoDB Clustered (Modelo Documental / CP): Base de datos NoSQL documental con esquema flexible BSON, índices secundarios e índices compuestos, y particionado horizontal nativo (sharding).

Decisión: Se selecciona MongoDB Clustered con un esquema de particionado (Sharding) por hash sobre la clave account_id.

- Posición en el Teorema CAP (Sistema CP): MongoDB se clasifica y configura formalmente como un sistema CP (Consistencia y Tolerancia a Particiones). Utiliza un Replica Set con un único nodo Primario que canaliza las escrituras. Se establecen políticas estrictas de writeConcern: 'majority' (la escritura solo se confirma cuando la mayoría de los nodos la han persistido en journal) y readConcern: 'majority' (las lecturas consumen exclusivamente datos confirmados por la mayoría, erradicando lecturas sucias).
- Rendimiento de Acceso: Al almacenar el perfil completo del cliente como un único documento BSON —evitando joins y accesos a múltiples particiones— las lecturas puntuales por la clave de particionado account_id se resuelven desde la caché en RAM del motor WiredTiger con latencia p50 inferior a 15 ms y p99 inferior a 50 ms. Es importante precisar el alcance de esta cifra: aplica a lecturas puntuales dirigidas al nodo primario, no a las escrituras. La política writeConcern: majority impone un costo adicional de confirmación en journal por parte de la mayoría del replica set (estimado en 20–40 ms), penalización que se

asume consciente y exclusivamente en la ruta asíncrona de actualización del perfil, nunca en la ruta síncrona de autorización.

Consecuencias: Se logra consistencia estricta para evitar la aprobación de transacciones fraudulentas concurrentes, consultas directas y atómicas por documento, y esquema flexible para agregar nuevas variables de riesgo sin migraciones. Como contrapartida (trade-off), en caso de una partición de red que aísle al nodo primario de la mayoría, el conjunto de réplicas entra temporalmente en proceso de elección de un nuevo primario (ventana de 2 a 4 segundos) durante la cual las escrituras en ese shard quedan suspendidas, sacrificando la disponibilidad temporal en favor de la integridad absoluta de los datos.

ADR 05: Elección del Orquestador y Modelado del DAG

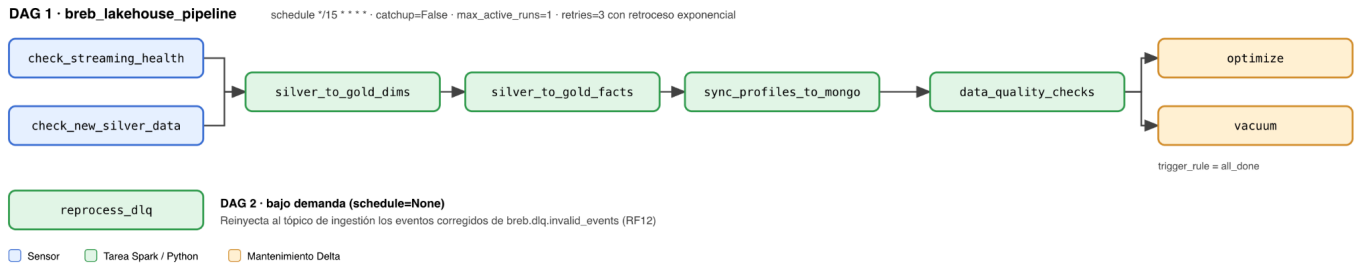
Contexto: Junto al streaming continuo (Jobs 1 a 3, que incluyen la limpieza de Bronze a Silver) existen procesos batch con dependencias estrictas que requieren coordinación explícita: consolidación del modelo dimensional en Gold con SCD Tipo 2, sincronización de perfiles hacia MongoDB, mantenimiento del Lakehouse (OPTIMIZE/VACUUM) y reproceso de la DLQ. No se puede construir la tabla de hechos antes de que Silver tenga datos nuevos y las dimensiones estén cargadas, y la ejecución manual es inviable en un sistema 24/7/365.

Alternativas consideradas:

- Cron del sistema operativo: Descartado. Dispara por tiempo, no por dependencias satisfechas: si Silver se retrasa por un pico de volumen, Gold se ejecuta sobre datos incompletos y produce una tabla de hechos silenciosamente corrupta. Sin reintentos, visibilidad ni backfill.
- Prefect 2.x: Mejor experiencia de desarrollo, pero integraciones con Spark menos maduras y menos operadores nativos para el stack elegido.
- Dagster: Su paradigma orientado a activos de datos es conceptualmente el más alineado con un Lakehouse por capas y se reconoce como la alternativa técnicamente más elegante. Descartado por curva de aprendizaje incompatible con el cronograma a S16 y por no ser la referencia del curso.
- Apache Airflow 2.8: DAGs en Python puro, estándar de facto y referencia del curso (S11).

Decisión: Se selecciona Apache Airflow 2.8 con LocalExecutor y metastore en PostgreSQL 15, por cuatro razones: (i) materializa las dependencias como aristas de un grafo —una tarea se dispara si y solo si sus upstream alcanzaron success—, satisfaciendo el RF9; (ii) operadores nativos para el stack elegido (SparkSubmitOperator, DockerOperator) y sensores para verificar precondiciones como el consumer lag del job de streaming; (iii) resiliencia declarativa vía retries=3 con retry_exponential_backoff, que operacionaliza el RNF-05, e idempotencia en reprocesos históricos combinando DAG runs parametrizados con MERGE INTO de Delta Lake; y (iv) un metastore que guarda el historial de ejecuciones de cada tarea (run, marca temporal y estado final), cubriendo la trazabilidad de procesos del RNF-06 sin código adicional.

Modelado del DAG: 'breb_lakehouse_pipeline', schedule cada 15 min, catchup=False y max_active_runs=1 para evitar escrituras concurrentes sobre la misma tabla Delta. El reproceso de la DLQ (RF12) se modela como un segundo DAG, reprocess_dlq, sin programación (schedule=None), que se dispara bajo demanda y reinyecta los eventos corregidos al tópico de ingestión.



El encadenamiento silver_to_gold_dims >> silver_to_gold_facts modela una dependencia de integridad referencial real: las dimensiones conformadas deben cargarse antes de la tabla de hechos, porque Fact_Transferencias resuelve las claves subrogadas de la versión SCD Tipo 2 vigente de Dim_Usuario.. Las tareas de mantenimiento usan trigger_rule='all_done' para compactar archivos pequeños incluso si una carga parcial falló. El job de streaming corre fuera del DAG como servicio permanente y Airflow sólo supervisa su salud, decisión deliberada porque Airflow planifica procesos finitos, no aplicaciones de larga duración.

Consecuencias: Se gana coordinación auditable, recuperación sin intervención manual y visibilidad del estado del pipeline para la demo del E2. Se asumen: (a) sobrecarga de ~2 GB de RAM por scheduler, webserver y metastore sobre un entorno local que ya aloja Kafka, Spark, MinIO y MongoDB, registrado como riesgo en la Sección 5; (b) granularidad mínima de programación de aproximadamente un minuto, razón adicional para delegar la baja latencia íntegramente a Spark Structured Streaming; y (c) acoplamiento del pipeline a Python y a versiones específicas de providers, con la deuda de mantenimiento correspondiente.

ADR 06: Plataforma de Despliegue — Docker Compose Local vs. Servicios Gestionados en la Nube

Contexto: Seis tecnologías distribuidas deben coexistir e interoperar bajo cuatro restricciones concretas: el E2 exige una demo en vivo de 8 minutos sin depender de conectividad ni credenciales institucionales; no hay presupuesto de infraestructura; los cuatro integrantes deben levantar un entorno idéntico y reproducible para desarrollar en paralelo; y el pipeline debe arrancar desde cero siguiendo el README.

Alternativas consideradas:

- AWS gestionado (MSK + EMR + S3 + MWAA + DocumentDB): La opción más cercana a producción, pero MSK y MWAA no tienen capa gratuita y el conjunto supera los 300 USD/mes. Riesgo adicional crítico: la sustentación quedaría dependiendo de la red del campus, y un fallo de conectividad comprometería el criterio de demo funcional (25% del bloque de presentación y defensa del E2) por causas ajenas a la calidad técnica.
- GCP (Pub/Sub + Dataproc + BigQuery + Composer): Descartada por coherencia arquitectónica: BigQuery es un warehouse propietario con almacenamiento acoplado, no un formato de tabla abierta, lo que contradice el ADR 03 y el requisito de Lakehouse.
- Databricks Community Edition: Spark y Delta gestionados sin costo, pero no permite desplegar Kafka, MongoDB ni Airflow, ni sostener jobs de streaming de larga duración; imposibilita la demo de extremo a extremo.

Docker Compose local con componentes open-source y MinIO como almacenamiento compatible con la API S3.

Decisión: Se selecciona despliegue local con Docker Compose, MinIO como capa de objetos y perfiles de recursos declarados por servicio. El argumento central no es la economía sino la portabilidad por paridad de interfaces: cada componente elegido es el mismo motor open-source que los servicios gestionados ejecutan por debajo, y MinIO implementa la API S3, por lo que migrar a producción se reduce a cambiar endpoints y credenciales —s3a:// apuntando a MinIO o a S3 real es idéntico código PySpark— sin reescribir lógica de negocio. Esto evita el vendor lock-in que ya motivó el descarte de Kinesis en el ADR 01, manteniendo coherencia en el conjunto de decisiones. Como beneficio adicional, el entorno local permite inducir fallos deliberadamente (detener un bróker, matar un ejecutor, aislar el primario de MongoDB) para validar empíricamente el RNF-05 y la ventana de elección descrita en el ADR 04, experimentos imposibles de provocar en servicios gestionados.

Consecuencias: Se obtiene reproducibilidad con un docker compose up, costo nulo, independencia de conectividad en la sustentación y experimentación destructiva controlada. Las contrapartidas se declaran explícitamente: (a) la cifra de 25.000 TPS del RNF-01 es un objetivo de diseño dimensionado analíticamente, no una medición —un entorno de 3 brokers en una sola máquina física no puede sostener ese caudal; las pruebas del E2 se ejecutarán en 500–1.000 eventos/segundo y la extrapolación se argumentó desde el modelo de particionado horizontal de Kafka, declarando el supuesto de linealidad como tal; (b) no es demostrable la elasticidad real ni la recuperación multizona,

por lo que la tolerancia a fallos se evidencia a nivel de contenedor y no de zona de disponibilidad; y (c) el stack completo demanda 12–16 GB de RAM, mitigado con perfiles de Compose que permiten levantar subconjuntos durante el desarrollo y un dataset sintético acotado.

Sección 5. Plan de Implementación

Esta sección establece la hoja de ruta hacia el Entregable 2 (S16), definiendo la distribución de tareas, los hitos intermedios con criterios de aceptación verificables y la matriz de riesgos técnicos con su plan de mitigación.

5.1 Distribución de Responsabilidades

Integrante	Rol	Componentes a implementar	Entregable verificable
Samuel Arango	Lead de Ingesta y Streaming	Productor sintético de eventos (Switch Bre-B y logs de App Nequi); clúster Kafka 3.6 en modo KRaft (3 brokers), tópico breb.transacciones.raw particionado por source_account y tópico DLQ; Job 1 (validación y enrutamiento) y Job 2 (ventanas deslizantes, score de riesgo y emisión de alertas)	Eventos fluyendo de extremo a extremo con latencia p99 medida y alertas publicadas en fraud.alerts.critical
Mateo Sanz Medina	Lead de Almacenamiento	Tablas Delta Lake Bronze, Silver y Gold sobre MinIO con particionado y Z-ORDER; Job 3 (saneamiento y deduplicación) y Job 4 (Star Schema con SCD Tipo 2 y MERGE por clave); colección user_risk_profiles en MongoDB con sharding por hash(account_id) y políticas majority	Las tres capas del Lakehouse consultables y el perfil de riesgo leído con p50 inferior a 15 ms
Nathalia Valentina Cardoza Azuaje	Lead de DataOps	docker-compose con perfiles de memoria, volúmenes y red; DAG breb_lakehouse_pipeline con SparkSubmitOperator, sensor de consumer lag y retries=3 con retroceso exponencial; tareas de mantenimiento (OPTIMIZE y VACUUM) y reproceso de la DLQ; README reproducible desde cero	Stack completo levantado con un solo comando y DAG ejecutándose en verde de punta a punta
Sebastian Andres Medina Cabezas	Lead de Consumo y Gobernanza	API REST en FastAPI que expone el score de riesgo desde MongoDB; tableros en Metabase que responden las cuatro preguntas de la Sección 1.3; trazabilidad por propagación de transaction_uuid, coordenadas de origen en Bronze y auditoría por Time Travel; instrumentación y medición de latencias p50 y p99 de todas las rutas	API respondiendo con p99 inferior a 300 ms, tablero operativo y evidencia reproducible de linaje

Contrato de integración. Para habilitar el trabajo en paralelo, en la primera semana de implementación se congelan dos interfaces que actúan como contrato entre los cuatro frentes: el esquema del evento publicado en breb.transacciones.raw y el esquema del documento user_risk_profiles. Cualquier modificación posterior requiere acuerdo de los cuatro integrantes.

5.2 Cronograma de Hitos Intermedios

Semana	Hito	Criterio de aceptación

S13	Infraestructura base operativa	docker compose up levanta Kafka, MinIO, Spark, MongoDB y Airflow (con el perfil de desarrollo); el productor sintético publica en breb.transacciones.raw y los mensajes se consumen correctamente. Esquemas del contrato de integración congelados.
S14	Ruta de streaming completa	Un evento publicado en Kafka se materializa en Bronze, en Silver y en user_risk_profiles; el Job 2 calcula el score y publica alertas. Latencia de extremo a extremo medida y por debajo de 3 segundos en p99 (RNF-02).
S15	Capa Gold, orquestación y consumo	El Job 4 construye Fact_Transferencias y las cuatro dimensiones; el DAG completo se ejecuta en verde respetando dependencias; la API de FastAPI y el tablero de Metabase responden al menos una de las cuatro preguntas de negocio de la Sección 1.3.
S16	Endurecimiento, ensayo y sustentación	Pruebas de fallo deliberado ejecutadas (detención de un bróker, terminación de un ejecutor de Spark y aislamiento del primario de MongoDB) para evidenciar el RNF-05; README validado levantando el pipeline desde cero en una máquina limpia; ensayo completo de la demo el día anterior; cada integrante capaz de responder preguntas sobre cualquier componente del sistema.

5.3 Matriz de Riesgos Técnicos y Mitigación

#	Riesgo	Prob. / Impacto	Mitigación	Responsable
R1	Saturación de memoria en los jobs de streaming por crecimiento del estado de las ventanas	Media / Alto	Watermark de 2 minutos en el Job 2 y de 10 minutos en el Job 3; limitación de maxOffsetsPerTrigger; checkpoint en MinIO y afinamiento de executor.memory	Samuel Arango
R2	Acumulación de archivos pequeños en Delta Lake por las micro-escrituras del streaming	Alta / Medio	Tarea de OPTIMIZE programada en el DAG cada 15 minutos y VACUUM con retención de 7 días; Z-ORDER por source_account en Silver, según la ficha técnica.	Mateo Sanz Medina
R3	Desalineación o corrupción del esquema en los JSON de entrada	Media / Medio	Validación contra esquema formal en el Job 1, enrutamiento de los inválidos a breb.dlq.invalid_events y tarea de reproceso bajo demanda sin generar duplicados (RF2 y RF12)	Samuel Arango
R4	Caída de un bróker de Kafka durante las pruebas de carga	Baja / Alto	Factor de replicación 3, acks=all y min.insync.replicas=2; la pérdida de un bróker se induce deliberadamente en S16 como evidencia del RNF-05	Samuel Arango y Nathalia Cardoza
R5	Agotamiento de la memoria del equipo anfitrión, dado que el stack completo demanda entre 12 y 16 GB de RAM, de los cuales Airflow aporta unos 2 GB (scheduler, webserver y metastore)	Alta / Alto	Perfiles de docker-compose que permiten levantar subconjuntos del stack durante el desarrollo; dataset sintético acotado; un único bróker con factor de replicación 1 en el perfil de desarrollo y los tres únicamente para la prueba de tolerancia a fallos	Nathalia Cardoza
R6	Conflicto de escritura entre el Job 2 y Airflow sobre el mismo documento de user_risk_profiles	Baja / Alto	Operaciones $set restringidas a subdocumentos disjuntos según el RF8, verificadas con una prueba de escritura concurrente antes de S15	Mateo Sanz Medina

El riesgo de no alcanzar el caudal de 25.000 eventos por segundo en un entorno local se encuentra declarado y acotado en las consecuencias del ADR 06, donde se establece que dicha cifra es un objetivo de diseño dimensionado analíticamente y que las pruebas del Entregable 2 se ejecutarán en un rango de 500 a 1.000 eventos por segundo.